Roboterdynamik – Praktikum (Modul MW2339)

Direkte Kinematik

Ziel dieser Übung ist es die Direkte Kinematik für Industrieroboter mit Drehgelenken zu implementieren. Der Roboter GoFa von ABB, siehe Abb. 1, dient uns als Beispiel. Zunächst müssen die DH-Parameter des Roboters bestimmt werden und in eine Konfigurationsdatei geschrieben werden. Dann muss die rekursive Berechnung der Positionen implementiert werden. Die grundlegende Struktur ist bereits als Python-Code vorhanden.



Abbildung 1: Sechs-achsiger Industrieroboter GoFa von ABB (©ABB)

Python ist nicht die passende Programmiersprache für harte Echtzeitanwendungen, ermöglicht aber im Rahmen eines Praktikums die gleichen Algorithmen mit etwas weniger Aufwand zu implementieren, als mit C++. Da die Nutzerschnittstelle zu vielen Robotern nicht sonderlich hochfrequent ist, reicht es sogar oft für Anwendungen aus. Für das Praktikum wird mindestens Python Version 3.10 benötigt. Wer sich mit der Notation von Matlab auskennt, kann folgende Vergleichstabelle verwenden, um die Notation für numpy zu lernen. Diese Bibliothek wird in python normalerweise und auch bei uns für Matrizen verwendet wird. https://numpy.org/doc/stable/user/numpy-for-matlab-users.html

Das Skript direct_kinematics.py erzeugt ein Roboterobjekt und führt daran beispielhaft die DK-Funktionen aus. Wir betrachten zunächst nur die Positionenebene und lassen DK-Funktionen für deren Ableitungen auskommentiert. Für das Beispiel werden alle sechs Achsen des Roboters nacheinander von einer Ausgangslage (0°) in eine Endlage (45°) bewegt. Die berechneten Positionen und Orientierungen aller Körper werden dann in eine Ausgabedatei trajectory .csv geschrieben. Diese Informationen sind z.B. für einen Viewer des Roboters wichtig, der die 3D-Modelle der einzelnen Robotersegmente passend im Raum anordnen muss. Hier im Praktikum können Sie die Datei auf folgender Webseite hochladen, die uns als Viewer dient:
https://web.am.ed.tum.de/roboterdynamik_praktikum

Machen Sie sich zunächst mit dem Skript vertraut, bevor Sie mit der Bearbeitung der Übung beginnen.

Für eine erfolgreiche Teilnahme am Praktikum müssen Sie die verwendeten Methoden und den Code (sowohl die von Ihnen ergänzten Stellen, also auch den Rest in diesem Projekt) erklären können. Machen Sie sich deshalb immer mit dem gesamten Code vertraut, der zu einem Termin verwendet wird, auch wenn Sie ihn nicht selbst geschrieben haben!

1 Denavit-Hartenberg Konvention

Um die Kinematik des Roboters in unseren Code einlesen zu können, wird diese nach der Denavit-Hartenberg Konvention aus der Vorlesung Roboterdynamik parametriert. Bestimmen Sie die Parameter anhand der Abb. 2 und tragen Sie diese in die Konfigurationsdatei robot_config_files/ABB_GoFa.yaml ein. Es gibt eine Undeutigkeit aufgrund zweier sich schneidender Achsen. Wählen Sie erst einmal eine Möglichkeit aus.

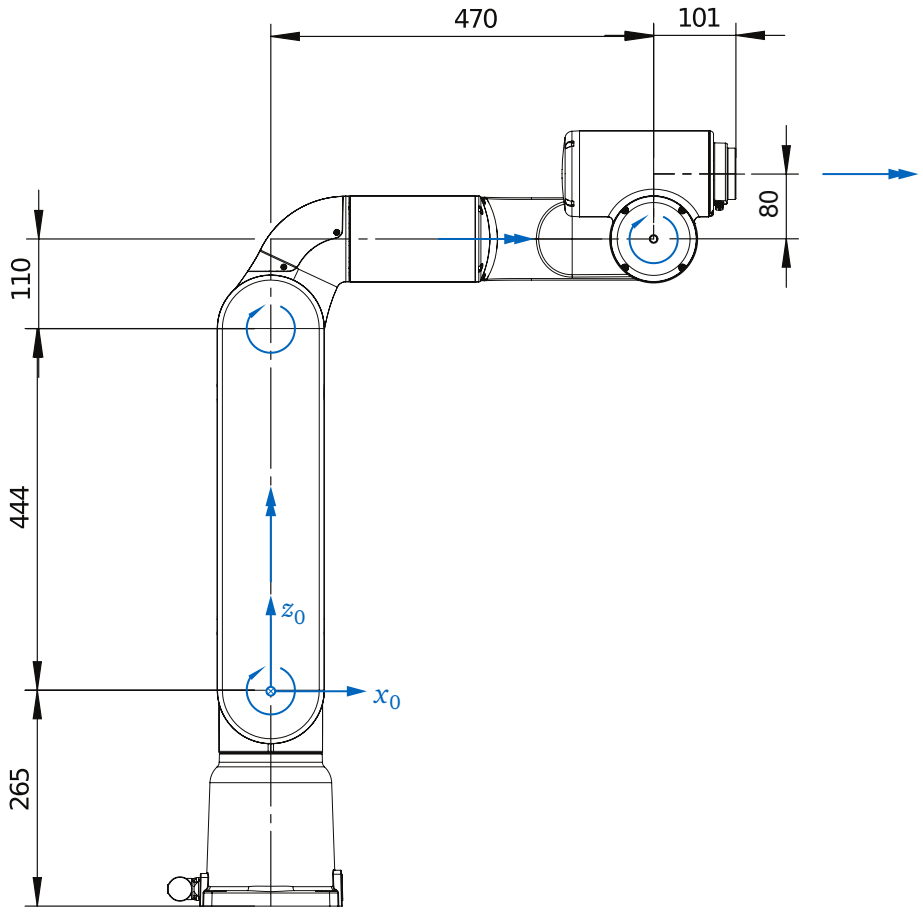


Abbildung 2: Abmessungen des Roboters und positive Drehrichtungen der Gelenke, Roboter befindet sich in $\boldsymbol{q} = \begin{bmatrix} 0 & -\pi/2 & 0 & 0 & 0 & 0 \end{bmatrix}^T$ (nach [**PM_CRB_15000-en**] mit blauen Annotationen)

2 Direkte Kinematik – Position

Wie im Modul Roboterdynamik gelernt, soll die Position des Tool Center Points rekursiv berechnet werden. Ergänzen Sie hierfür die notwendigen Funktionen in den Klassen Robot und Link, die beide in der Dateien rdplib/robot.py implementiert sind.

Überprüfen Sie Ihr Ergebnis mit dem Viewer! Stimmt Ihre Wahl der Koordinatensysteme mit der Konfiguration in Abb. 2 überein? Ändern Sie im Zweifel Ihre Parameter (an uneindeutigen Stellen).

Wenn Sie auf Probleme stoßen, versuchen Sie z.B. einige Körper in der Konfigurationsdatei auszukommentieren. So ist das System leichter nachzuvollziehen und Sie finden Ihren Fehler schneller.

3 Direkte Kinematik – Geschwindigkeit

Neben der Berechnung von Positionen und Orientierungen gehört auch die Bestimmung von Geschwindigkeiten zur *direkten Kinematik*. Ergänzen Sie die nötigen Lücken und kommentieren Sie die entsprechenden Zeilen in direct_kinematics.py ein.

4 Direkte Kinematik – Beschleunigung

Ergänzen Sie die nötigen Lücken und kommentieren Sie die entsprechenden Zeilen in direct_kinematics.py ein.

5 Direkte Kinematik – Jacobi-Matrizen

Ergänzen Sie die nötigen Lücken und kommentieren Sie die entsprechenden Zeilen in direct_kinematics.py ein.

6 Bonus, keine Pflicht – Überprüfen der Ergebnisse mit Unittests

In Codeprojekten empfiehlt es sich die programmierten Module bald auf Ihre Korrektheit hin zu prüfen. So kann man schon früh Fehler finden und spart sich einiges Debugging im späteren Verlauf des Projekts. Die Idee ist, dass man einzelne Bausteine überprüft, und so bei Fehlern nicht an so vielen Stellen suchen muss. Sind die Bausteine einmal miteinander zu einem großen Ganzen verknüpft und ergeben nicht das erwartete Ergebnis, kann der Fehler in alle Bausteinen zu finden sein. Das verlängert tendenziell die Debuggingzeit.

In Python gibt es ein integriertes unittest Modul (https://docs.python.org/3/library/unittest.html). Führt man das Modul aus

```
python3 -m unittest
```

so werden alle Tests durchgeführt, die das Modul finden kann. Eine einzelne Testfunktion kann man durch den folgenden Befehl ausführen:

```
python3 -m unittest testing.testDK.TestDK.test_velocity_calculation
```

Im Ordner testing in der Datei testDK gibt es eine Klasse TestDK, deren Funktionen nun ausgeführt werden. Man kann auch eine einzelne Testfunktion mit einem weiteren Punkt hinten anhängen. Damit eine Klasse eine Testklasse ist, erbt sie von unittest.TestCase. Ob ein Testfall erfolgreicht ist oder nicht hängt an den Überprüfungsfunktionen wie z.B. von unittest assertEqual oder von numpy np.testing.assert_allclose.

Wenn Sie jetzt die Testfunktion für die Geschwindigkeit aufrufen, wird der Testfall scheitern. Ergänzen Sie die Funktion so, dass Sie damit Ihre Geschwindigkeitsberechnung kontrollieren können. Für die Beschleunigung und die Jacobimatrizen können weitere Funktionen hinzugefügt werden.